Searching major star catalogs and Simbad by position or name with vo_conesearch

Authors

P. L. Lim

Learning Goals

- Perform a cone search around M31 using a web service.
- Write the result out to a LaTeX table.
- Perform a SIMBAD query using the cone search result.
- Extract metadata from the cone search catalog.
- Sort cone search results by angular distance.
- Search multiple cone search services at once (synchronously and asynchronously).
- Estimate the run time of a cone search.

Keywords

astroquery, table, coordinates, units, vo_conesearch, LaTex, SIMBAD, matplotlib

Summary

This tutorial desmonstrates the Cone Search subpackage, which allows you to query a catalog of astronomical sources and obtain those that lie within a cone of a given radius around the given

position.

Imports

```
# Python standard library
import time
import warnings

# Third-party software

# Astropy
from astropy import coordinates as coord
from astropy import units as u
from astropy.table import Table

# Astroquery. This tutorial requires 0.3.5 or greater.
import astroquery
from astroquery.simbad import Simbad
from astroquery.vo_conesearch import conf, conesearch, vos_catalog

# Set up matplotlib
%matplotlib inline
```

If you are running an older version of `astroquery`, you might need to set `vos_baseurl` yourself, as follows.

```
from astropy.utils import minversion

if not minversion(astroquery, "0.3.10"):
    conf.vos_baseurl = "https://astroconda.org/aux/vo_databases/"
```

To start, it might be useful to list the available Cone Search catalogs first. By default, catalogs that pass nightly validation are included. Validation is hosted by Space Telescope Science Institute (STScI).

```
conesearch.list_catalogs()
```

```
---------------------------------------------------------------------------
SSLCertVerificationError                  Traceback (most recent call last)
File /opt/hostedtoolcache/Python/3.12.10/x64/lib/python3.12/urllib/request.py:1344,
   1343 try:
-> 1344     h.request(req.get_method(), req.selector, req.data, headers,
   1345               encode_chunked=req.has_header('Transfer-encoding'))
   1346 except OSError as err: # timeout error

File /opt/hostedtoolcache/Python/3.12.10/x64/lib/python3.12/http/client.py:1338, in
   1337 """Send a complete request to the server."""
-> 1338 self._send_request(method, url, body, headers, encode_chunked)

File /opt/hostedtoolcache/Python/3.12.10/x64/lib/python3.12/http/client.py:1384, in
   1383     body = _encode(body, 'body')
-> 1384 self.endheaders(body, encode_chunked=encode_chunked)

File /opt/hostedtoolcache/Python/3.12.10/x64/lib/python3.12/http/client.py:1333, in
   1332     raise CannotSendHeader()
-> 1333 self._send_output(message_body, encode_chunked=encode_chunked)

File /opt/hostedtoolcache/Python/3.12.10/x64/lib/python3.12/http/client.py:1093, in
   1092 del self._buffer[:]
-> 1093 self.send(msg)
   1095 if message_body is not None:
   1096
   1097     # create a consistent interface to message_body

File /opt/hostedtoolcache/Python/3.12.10/x64/lib/python3.12/http/client.py:1037, in
   1036 if self.auto_open:
-> 1037     self.connect()
   1038 else:

File /opt/hostedtoolcache/Python/3.12.10/x64/lib/python3.12/http/client.py:1479, in
   1477     server_hostname = self.host
-> 1479 self.sock = self._context.wrap_socket(self.sock,
   1480                                          server_hostname=server_hostname)

File /opt/hostedtoolcache/Python/3.12.10/x64/lib/python3.12/ssl.py:455, in SSLContex
    449 def wrap_socket(self, sock, server_side=False,
    450                 do_handshake_on_connect=True,
    451                 suppress_ragged_eofs=True,
    452                 server_hostname=None, session=None):
    453     # SSLSocket class handles server_hostname encoding before it calls
    454     # ctx._wrap_socket()
--> 455     return self.sslsocket_class._create(
    456         sock=sock,
    457         server_side=server_side,
    458         do_handshake_on_connect=do_handshake_on_connect,
    459         suppress_ragged_eofs=suppress_ragged_eofs,
    460         server_hostname=server_hostname,
    461         context=self,
    462         session=session
    463     )
```

```
File /opt/hostedtoolcache/Python/3.12.10/x64/lib/python3.12/ssl.py:1041, in SSLSocke
   1040                 raise ValueError("do_handshake_on_connect should not be spec
-> 1041             self.do_handshake()
   1042 except:

File /opt/hostedtoolcache/Python/3.12.10/x64/lib/python3.12/ssl.py:1319, in SSLSocke
   1318         self.settimeout(None)
-> 1319     self._sslobj.do_handshake()
   1320 finally:

SSLCertVerificationError: [SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed

During handling of the above exception, another exception occurred:

URLError                                  Traceback (most recent call last)
File /opt/hostedtoolcache/Python/3.12.10/x64/lib/python3.12/site-packages/astropy/ut
   1245 try:
-> 1246     return urlopener.open(req, timeout=timeout)
   1247 except urllib.error.URLError as exc:

File /opt/hostedtoolcache/Python/3.12.10/x64/lib/python3.12/urllib/request.py:515, i
    514 sys.audit('urllib.Request', req.full_url, req.data, req.headers, req.get_met
--> 515 response = self._open(req, data)
    517 # post-process response

File /opt/hostedtoolcache/Python/3.12.10/x64/lib/python3.12/urllib/request.py:532, i
    531 protocol = req.type
--> 532 result = self._call_chain(self.handle_open, protocol, protocol +
    533                           '_open', req)
    534 if result:

File /opt/hostedtoolcache/Python/3.12.10/x64/lib/python3.12/urllib/request.py:492, i
    491 func = getattr(handler, meth_name)
--> 492 result = func(*args)
    493 if result is not None:

File /opt/hostedtoolcache/Python/3.12.10/x64/lib/python3.12/urllib/request.py:1392,
   1391 def https_open(self, req):
-> 1392     return self.do_open(http.client.HTTPSConnection, req,
   1393                         context=self._context)

File /opt/hostedtoolcache/Python/3.12.10/x64/lib/python3.12/urllib/request.py:1347,
   1346 except OSError as err: # timeout error
-> 1347     raise URLError(err)
   1348 r = h.getresponse()

URLError: <urlopen error [SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed:

During handling of the above exception, another exception occurred:

URLError                                  Traceback (most recent call last)
Cell In[3], line 1
----> 1 conesearch.list_catalogs()
```

```
File /opt/hostedtoolcache/Python/3.12.10/x64/lib/python3.12/site-packages/astroquery
    330 def list_catalogs(**kwargs):
    331     """
    332     Return the available Cone Search catalogs as a list of strings.
    333     These can be used for the ``catalog_db`` argument to
   (...)    360
    361     """
--> 362     return vos_catalog.list_catalogs(conf.conesearch_dbname, **kwargs)

File /opt/hostedtoolcache/Python/3.12.10/x64/lib/python3.12/site-packages/astroquery
    905 def list_catalogs(service_type, cache=True, verbose=True, **kwargs):
    906     """List the catalogs available for the given service type.
    907
    908     Parameters
   (...)    935
    936     """
--> 937     return get_remote_catalog_db(service_type, cache=cache,
    938                                  verbose=verbose).list_catalogs(**kwargs)

File /opt/hostedtoolcache/Python/3.12.10/x64/lib/python3.12/site-packages/astroquery
    631 def get_remote_catalog_db(dbname, *, cache=True, verbose=True):
    632     """
    633     Get a database of VO services (which is a JSON file) from a remote
    634     location.
   (...)    654
    655     """
--> 656     return VOSDatabase.from_json(
    657         urljoin_keep_path(conf.vos_baseurl, dbname + '.json'),
    658         encoding='utf8', cache=cache, show_progress=verbose)

File /opt/hostedtoolcache/Python/3.12.10/x64/lib/python3.12/site-packages/astroquery
    487 @classmethod
    488 def from_json(cls, filename, **kwargs):
    489     """
    490     Create a database of VO services from a JSON file.
    491
   (...)    522
    523     """
--> 524     with get_readable_fileobj(filename, **kwargs) as fd:
    525         tree = json.load(fd)
    527     return cls(tree)

File /opt/hostedtoolcache/Python/3.12.10/x64/lib/python3.12/contextlib.py:137, in _G
    135 del self.args, self.kwds, self.func
    136 try:
--> 137     return next(self.gen)
    138 except StopIteration:
    139     raise RuntimeError("generator didn't yield") from None

File /opt/hostedtoolcache/Python/3.12.10/x64/lib/python3.12/site-packages/astropy/ut
    354 is_url = _is_url(name_or_obj)
    355 if is_url:
--> 356     name_or_obj = download_file(
```

```
    357         name_or_obj,
    358         cache=cache,
    359         show_progress=show_progress,
    360         timeout=remote_timeout,
    361         sources=sources,
    362         http_headers=http_headers,
    363     )
    364 fileobj = io.FileIO(name_or_obj, "r")
    365 if is_url and not cache:
```

File /opt/hostedtoolcache/Python/3.12.10/x64/lib/python3.12/site-packages/astropy/ut

```
   1578     raise KeyError(
   1579         f"No sources listed and file {remote_url} not in cache! "
   1580         "Please include primary URL in sources if you want it to be "
   1581         "included as a valid source."
   1582     )
   1583 elif len(sources) == 1:
-> 1584     raise errors[sources[0]]
   1585 else:
   1586     raise urllib.error.URLError(
   1587         f"Unable to open any source! Exceptions were {errors}"
   1588     ) from errors[sources[0]]
```

File /opt/hostedtoolcache/Python/3.12.10/x64/lib/python3.12/site-packages/astropy/ut

```
   1546 for source_url in sources:
   1547     try:
-> 1548         f_name = _download_file_from_source(
   1549             source_url,
   1550             timeout=timeout,
   1551             show_progress=show_progress,
   1552             cache=cache,
   1553             remote_url=remote_url,
   1554             pkgname=pkgname,
   1555             http_headers=http_headers,
   1556             ssl_context=ssl_context,
   1557             allow_insecure=allow_insecure,
   1558         )
   1559         # Success!
   1560         break
```

File /opt/hostedtoolcache/Python/3.12.10/x64/lib/python3.12/site-packages/astropy/ut

```
   1329         else:
   1330             raise
-> 1332 with _try_url_open(
   1333     source_url,
   1334     timeout=timeout,
   1335     http_headers=http_headers,
   1336     ftp_tls=ftp_tls,
   1337     ssl_context=ssl_context,
   1338     allow_insecure=allow_insecure,
   1339 ) as remote:
   1340     info = remote.info()
   1341     try:
```

```
File /opt/hostedtoolcache/Python/3.12.10/x64/lib/python3.12/site-packages/astropy/ut
   1261 if not allow_insecure:
   1262     msg += (
   1263         " or in both cases you can work around this by "
   1264         "passing allow_insecure=True, but only if you "
   1265         "understand the implications; the original error "
   1266         f"was: {reason}"
   1267     )
-> 1268     raise urllib.error.URLError(msg)
   1269 else:
   1270     msg += ". Re-trying with allow_insecure=True."

URLError: <urlopen error Verification of TLS/SSL certificate at https://astroconda.c
```

Next, let's pick an astronomical object of interest. For example, M31.

```
c = coord.SkyCoord.from_name("M31", frame="icrs")
print(c)
```

By default, a basic Cone Search goes through the list of catalogs and *stops* at the first one that returns non-empty VO table. Let's search for objects within 0.1 degree around M31. You will see a lot of warnings that were generated by VO table parser but ignored by Cone Search service validator. VO compliance enforced by Cone Search providers is beyond the control of `astroquery.vo_conesearch` package.

The result is an [Astropy table](#).

```
result = conesearch.conesearch(c, 0.1 * u.degree)
```

```
print("First non-empty table returned by", result.url)
print("Number of rows is", len(result))
```

```
print(result)
```

This table can be manipulated like any other Astropy table; e.g., re-write the table into LaTeX format.

```
result.write("my_result.tex", format="ascii.latex", overwrite=True)
```

You can now use your favorite text editor to open the `my_result.tex` file, but here, we are going to read it back into another Astropy table.

Note that the extra `data_start=4` option is necessary due to the non-roundtripping nature of LaTeX reader/writer (see astropy issue 5205).

```
result_tex = Table.read("my_result.tex", format="ascii.latex", data_start=4)
print(result_tex)
```

Cone Search results can also be used in conjuction with other types of queries. For example, you can query SIMBAD for the first entry in your result above.

```
# Due to the unpredictability of external services,
# The first successful query result (above) might differ
# from run to run.
#
# CHANGE THESE VALUES to the appropriate RA and DEC
# column names you see above, if necessary.
# These are for http://gsss.stsci.edu/webservices/vo/ConeSearch.aspx?CAT=GSC23&
ra_colname = "ra"
dec_colname = "dec"
```

```
# Don't run this cell if column names above are invalid.
if ra_colname in result.colnames and dec_colname in result.colnames:
    row = result[0]
    simbad_obj = coord.SkyCoord(
        ra=row[ra_colname] * u.deg, dec=row[dec_colname] * u.deg
    )
    print("Searching SIMBAD for\n{}\n".format(simbad_obj))
    simbad_result = Simbad.query_region(simbad_obj, radius=5 * u.arcsec)
    print(simbad_result)
else:
    print(
        "{} or {} not in search results. Choose from: {}".format(
            ra_colname, dec_colname, " ".join(result.colnames)
        )
    )
```

Now back to Cone Search… You can extract metadata of this Cone Search catalog.

```
my_db = vos_catalog.get_remote_catalog_db(conf.conesearch_dbname)
try:
    my_cat = my_db.get_catalog_by_url(result.url)
except AttributeError:
    my_cat = my_db.get_catalog_by_url(result.url + "&")
print(my_cat.dumps())
```

If you have a favorite catalog in mind, you can also perform Cone Search only on that catalog. A list of available catalogs can be obtained by calling `conesearch.list_catalogs()`, as mentioned above.

```
try:
    result = conesearch.conesearch(
        c, 0.1 * u.degree, catalog_db="The USNO-A2.0 Catalogue (Monet+ 1998) 1"
    )
except Exception:
    # We provide a cached version of the result table in case the query fails
    # due to an intermittent server-side issue, or if you do not have an
    # internet connection
    result = Table.read("usno-A2-result.fits")
```

```
print("Number of rows is", len(result))
```

Let's explore the 3 rows of astronomical objects found within 0.1 degree of M31 in the given catalog and sort them by increasing distance. For this example, the VO table has several columns that might include:

- `_r` = Angular distance (in degrees) between object and M31
- `USNO-A2.0` = Catalog ID of the object
- `RAJ2000` = Right ascension of the object (epoch=J2000)
- `DEJ2000` = Declination of the object (epoch=J2000)

Note that column names, meanings, order, etc. might vary from catalog to catalog.

```
col_names = result.colnames
print(col_names)
```

```
# Before sort
print(result)
```

```
# After sort
result.sort("_r")
print(result)
```

You can also convert the distance to arcseconds.

```
result["_r"].to(u.arcsec)
```

What if you want *all* the results from *all* the catalogs? And you also want to suppress all the VO table warnings and informational messages?

Warning: This can be time and resource intensive.

```
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    all_results = conesearch.search_all(c, 0.1 * u.degree, verbose=False)
```

```
for url, tab in all_results.items():
    print(url, "returned", len(tab), "rows")
```

```
# Pick out the first one with "I/220" in it.
i220keys = [k for k in all_results if "I/220" in k]
my_favorite_result = all_results[i220keys[0]]
print(my_favorite_result)
```

Asynchronous Searches

Asynchronous versions (i.e., search will run in the background) of `conesearch()` and `search_all()` are also available. Result can be obtained using the asynchronous instance's `get()` method that returns the result upon completion or after a given `timeout` value in seconds.

```
try:
    async_search = conesearch.AsyncConeSearch(
        c, 0.1 * u.degree, catalog_db="The USNO-A2.0 Catalogue (Monet+ 1998) 1"
    )
    print("Am I running?", async_search.running())

    time.sleep(3)
    print("After 3 seconds. Am I done?", async_search.done())
    print()

    result = async_search.get(timeout=30)
    print("Number of rows returned is", len(result))

except Exception:
    # We provide a cached version of the result table in case the query fails
    # due to an intermittent server-side issue, or if you do not have an
    # internet connection
    result = Table.read("usno-A2-result.fits")
```

```
async_search_all = conesearch.AsyncSearchAll(c, 0.1 * u.degree)
print("Am I running?", async_search_all.running())
print("Am I done?", async_search_all.done())
print()

all_results = async_search_all.get(timeout=60)
for url, tab in all_results.items():
    print(url, "returned", len(tab), "rows")
```

Estimating the Search Time

Let's predict the run time of performing Cone Search on `http://gsss.stsci.edu/webservices/vo/ConeSearch.aspx?CAT=GSC23&` with a radius of 0.1 degrees. For now, the prediction assumes a very simple linear model, which might or might not reflect the actual trend.

This might take a while.

```
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    t_est, n_est = conesearch.predict_search(
        "http://gsss.stsci.edu/webservices/vo/ConeSearch.aspx?CAT=GSC23&",
        c,
        0.1 * u.degree,
        verbose=False,
        plot=True,
    )
```

```
print("Predicted run time is", t_est, "seconds")
print("Predicted number of rows is", n_est)
```

Let's get the actual run time and number of rows to compare with the prediction above. This might take a while.

As you will see, the prediction is not spot on, but it's not too shabby (at least, not when we tried it!). Note that both predicted and actual run time results also depend on network latency and responsiveness of the service provider.

```
t_real, tab = conesearch.conesearch_timer(
    c,
    0.1 * u.degree,
    catalog_db="http://gsss.stsci.edu/webservices/vo/ConeSearch.aspx?CAT=GSC23&",
    verbose=False,
)
```

```
print("Actual run time is", t_real, "seconds")
print("Actual number of rows is", len(tab))
```